

**INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY
(ICDFA)**

CIP-A104: Offensive Security

**Assignment 4: Lab 4: XSS, Session Management and Authentication Resilience
Assessment**

Student Name: HABEEBAH ADEYINKA OLUGBOGI

Registration Number: c11.ehit2617337@icdfa.edu.ng

Cohort Program: ETHICAL HACKER INTERNSHIP

Instructor:

Submission Date: 03/09/2026

Executive Summary

This document covers an allowed test of cross site scripting, session handling, and login protection. The work was done in a separate Kali Linux virtual machine. Only local copies of DVWA and OWASP Mutillidae II were used. The XSS checks looked at reflected, DOM based, and stored cases. I tested both apps with the smallest security level they offer. I used a harmless marker such as LAB4_XSS_2026 and small related changes. When the browser ran the marker, it showed alert pop ups. I also saved the request and response logs to show what happened. After that, I raised the security setting for each app. In every case, the visible result went away. This happened due to output encoding and input checks, or a mix of both. Even so, the core design stayed the same. User data was still put into HTML and the page DOM. In one place, that data still reached a SQL statement by concatenation. Next came session tests. Neither app swapped the session id at sign in. At logout, both apps did end the session on the server side, which stopped the old session from working. I then reviewed the cookies. None of the cookies had the Secure flag in either app. I also saw uneven use of HttpOnly and SameSite. Some cookies had those flags and others did not. For login strength, I tried five wrong logins on one test account. There was no lockout and no added waiting time. No CAPTCHA showed up either. Based on this run, there was no real brute force defense at the settings used.

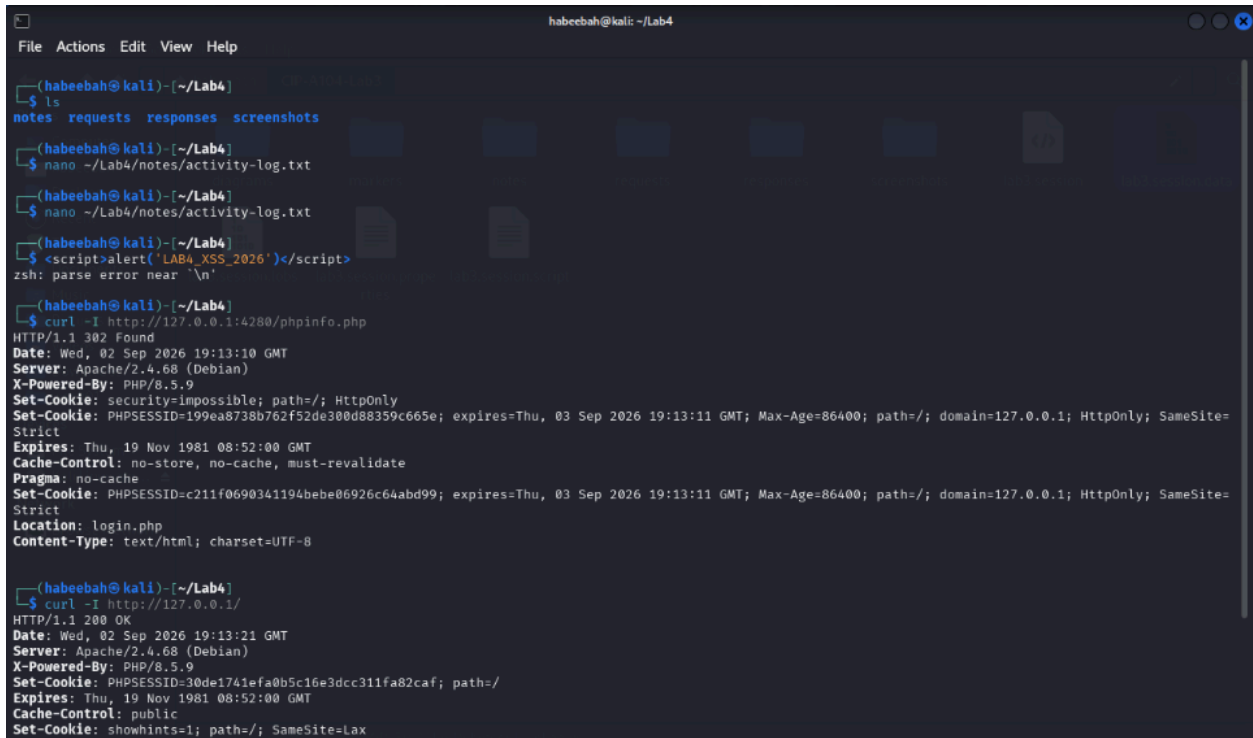
Overall, when the security level was raised, the symptoms were reduced. The trust boundary issues were still there. The fix plan in the next part lists root cause changes for each problem.

Phase 1: Scope, tools, and setup

Tests ran from a Kali Linux virtual machine. The targets were only local DVWA at 127.0.0.1:4280 and OWASP Mutillidae II at 127.0.0.1. All traffic went through Burp Suite and OWASP ZAP. This let us save each request and response.

To keep results easy to track, a marker name was used in every proof. The main marker was LAB4_XSS_2026. In cases where a script tag would not fit, other plain text markers were used. Examples include LAB4_DOM_2026 and LAB4-STORED-2026.

Before any real checks, baseline curl -I calls were done. Those checks showed both services were up and matched what was expected.



```
habeebah@kali: ~/Lab4
File Actions Edit View Help

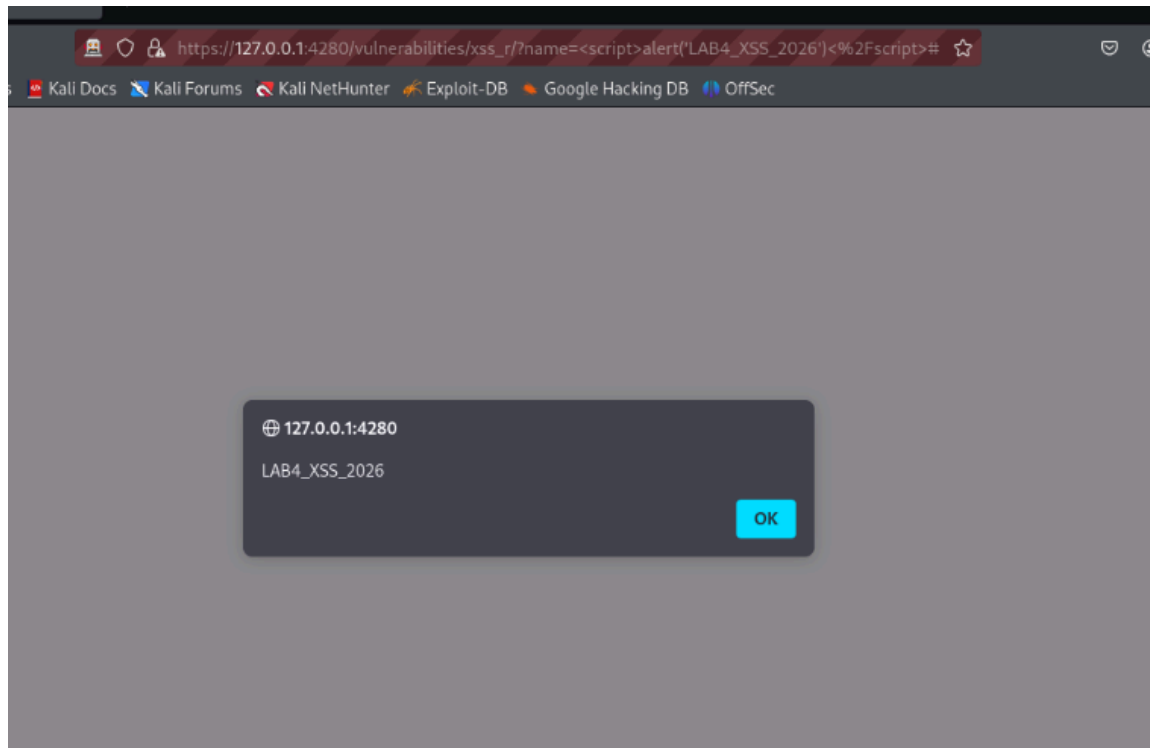
(habeebah@kali)~-[~/Lab4]
$ ls
notes requests responses screenshots
(habeebah@kali)~-[~/Lab4]
$ nano ~/Lab4/notes/activity-log.txt
(habeebah@kali)~-[~/Lab4]
$ nano ~/Lab4/notes/activity-log.txt
(habeebah@kali)~-[~/Lab4]
$ <script>alert('LAB4_XSS_2026')</script>
zsh: parse error near `\'
(habeebah@kali)~-[~/Lab4]
$ curl -I http://127.0.0.1:4280/phpinfo.php
HTTP/1.1 302 Found
Date: Wed, 02 Sep 2026 19:13:10 GMT
Server: Apache/2.4.68 (Debian)
X-Powered-By: PHP/8.5.9
Set-Cookie: security=impossible; path=/; HttpOnly
Set-Cookie: PHPSESSID=199ea8738b762f52de380d88359c665e; expires=Thu, 03 Sep 2026 19:13:11 GMT; Max-Age=86400; path=/; domain=127.0.0.1; HttpOnly; SameSite=Strict
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: no-store, no-cache, must-revalidate
Pragma: no-cache
Set-Cookie: PHPSESSID=c211f0690341194bebe06926c64abd99; expires=Thu, 03 Sep 2026 19:13:11 GMT; Max-Age=86400; path=/; domain=127.0.0.1; HttpOnly; SameSite=Strict
Location: login.php
Content-Type: text/html; charset=UTF-8

(habeebah@kali)~-[~/Lab4]
$ curl -I http://127.0.0.1/
HTTP/1.1 200 OK
Date: Wed, 02 Sep 2026 19:13:21 GMT
Server: Apache/2.4.68 (Debian)
X-Powered-By: PHP/8.5.9
Set-Cookie: PHPSESSID=30de1741efa0b5c16e3dccc311fa82caf; path=/
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: public
Set-Cookie: showhints=1; path=/; SameSite=Lax
```

Phase 2: DVWA reflected XSS at Low security

DVWA Security Level was set to Low. Then was sent in the Reflected XSS field called "What's your name?" An alert popped up. The alert text was LAB4_XSS_2026. This shows the browser ran the input as script when the page rendered it. It was not just text shown on the screen. ZAP request matched the sink behavior. The response showed the payload placed in the page without escaping. It appeared inside the page content as

Hello



Request:

```
GET
http://127.0.0.1:4280/vulnerabilities/xss_r/?name=%3Cscript%3Ealert%28%27LAB4_XSS_2026%27%29%3C%2Fscript%3E HTTP/1.1
Host: 127.0.0.1:4280
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Connection: keep-alive
Referer: https://127.0.0.1:4280/vulnerabilities/xss_r/
Cookie: security=low; PHPSESSID=7f12bclbde9588f1150466b077687813
Upgrade-Insecure-Requests: 1
Sec-Fetch-Dest: document
Sec-Fetch-Mode: navigate
Sec-Fetch-Site: same-origin
Sec-Fetch-User: ?1
Priority: u=0, i
```

Response:

```
HTTP/1.1 200 OK
Date: Wed, 02 Sep 2026 19:21:40 GMT
Server: Apache/2.4.68 (Debian)
X-Powered-By: PHP/8.5.9
Expires: Tue, 23 Jun 2009 12:00:00 GMT
Cache-Control: no-cache, must-revalidate
Pragma: no-cache
X-XSS-Protection: 0
Vary: Accept-Encoding
Content-Length: 4804
Keep-Alive: timeout=5, max=100

<p>What's your name?</p>
<input type="text" name="name">
<input type="submit" value="Submit">
</form>
<pre>Hello <script>alert('LAB4_XSS_2026')</script></pre>
</div>
```

ID	Source	Req. Timestamp	Method	URL	Code	Reason	RTT	Size Resp. Body	Highest Alert	Note	Tags
25	Pr...	02/09/2026, 20:20:58	GET	http://127.0.0.1:4280/favicon.ico	200	OK	2...	1,406 bytes			
26	Pr...	02/09/2026, 20:21:10	GET	http://127.0.0.1:4280/vulnerabilities/xss_r/...	200	OK	1...	4,804 bytes	Medium		Form, Script
27	Pr...	02/09/2026, 20:21:13	GET	http://127.0.0.1:4280/favicon.ico	200	OK	1...	1,406 bytes			
28	Pr...	02/09/2026, 20:21:34	GET	http://127.0.0.1:4280/vulnerabilities/xss_r/...	200	OK	1...	4,748 bytes			Form, Script
29	Pr...	02/09/2026, 20:21:40	GET	http://127.0.0.1:4280/vulnerabilities/xss_r/...	200	OK	1...	4,804 bytes			Form, Script

Phase 3: DVWA reflected XSS at Impossible security

The same input was sent again. This time DVWA Security Level was set to Impossible. No alert showed up. Instead, the page printed the characters as plain text. It showed Hello on the page. So the change across levels was about how the output was encoded at render time. The system still accepted the request, but it escaped the characters before they reached the page view.

Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

JavaScript Attacks

Authorisation Bypass

Open HTTP Redirect

Cryptography

API

DVWA Security

PHP Info

About

Logout

DVWA

Vulnerability: Reflected Cross Site Scripting (XSS)

What's your name?

Submit

Hello >

More Information

- <https://owasp.org/www-community/attacks/xss/>
- <https://owasp.org/www-community/xss-filter-evasion-cheatsheet>
- https://en.wikipedia.org/wiki/Cross-site_scripting
- <https://www.cgisecurity.com/xss-faq.html>
- <https://www.scriptalert1.com/>

Username: admin

Security Level: Possible

View Source

View Help

Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

JavaScript Attacks

Authorisation Bypass

Open HTTP Redirect

Cryptography

API

DVWA Security

PHP Info

About

Logout

DVWA

Vulnerability: Reflected Cross Site Scripting (XSS)

What's your name?

Submit

Hello <script>alert('LAB4_XSS_2026')</script>

More Information

- <https://owasp.org/www-community/attacks/xss/>
- <https://owasp.org/www-community/xss-filter-evasion-cheatsheet>
- https://en.wikipedia.org/wiki/Cross-site_scripting
- <https://www.cgisecurity.com/xss-faq.html>
- <https://www.scriptalert1.com/>

Username: admin

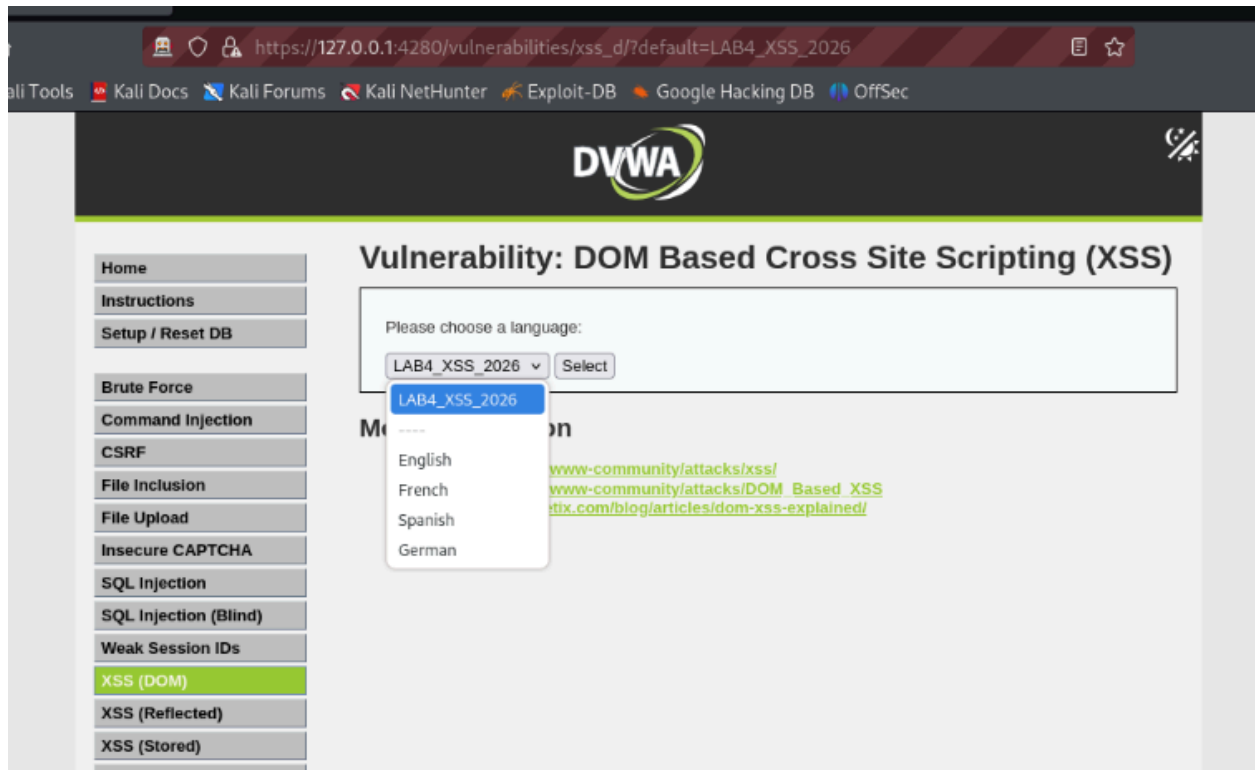
Security Level: Security Level: impossible

View Source

View Help

Phase 4: DVWA DOM XSS

With Low security, adding ?default=LAB4_XSS_2026 to the DOM XSS page made a value show up as a chosen item in the language dropdown. Firefox Inspector showed the value ended up in the page using client code that reads document.location.href. This lets us point to the URL parameter as the source, and to the DOM write behavior as the sink. Even so, this trial used a simple marker and not a script tag.



← → ↻ 🏠 127.0.0.1:4280/vulnerabilities/xss_d?default=LAB4_XSS_2026 📖 ⭐

Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB OffSec



Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

Vulnerability: DOM Based Cross Site Scripting (XSS)

Please choose a language:

LAB4_XSS_2026 Select

More Information

- <https://owasp.org/www-community/attacks/xss/>
- https://owasp.org/www-community/attacks/DOM_Based_XSS
- <https://www.acunetix.com/blog/articles/dom-xss-explained/>

Inspector Console Debugger Network Style Editor Performance Memory Storage Accessibility Application

LAB4_XSS_2026

<div class="vulnerable_code_area">
<p>Please choose a language:</p>
<form name="XSS" method="GET">
<select name="default">
><script>alert('XSS')</script>
<option value="LAB4_XSS_2026">LAB4_XSS_2026</option>
<option value="" disabled="disabled">----</option>
</select>
</form>
</div>

1 of 2 +

Filter Styles

element {
}
Inherited from select
input, textarea, select {
font: 100% arial,sans-serif;
}

main.css:32

Layout Computed

Flexbox
Select a Flex container or item

Grid
CSS Grid is not in use on this element



Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

JavaScript Attacks

Authorisation Bypass

Open HTTP Redirect

Cryptography

API

DVWA Security

PHP Info

About

Logout

Vulnerability: DOM Based Cross Site Scripting (XSS)

Please choose a language:

LAB4_XSS_2026 Select

LAB4_XSS_2026

English

French

Spanish

German

Username: admin

Security Level: Security Level: impossible

View Source View Help

With High security, the same URL parameter did not show anything in the dropdown. The dropdown also went back to its normal default. Inspector indicated the client code now decodes and checks the parameter before it uses it.



The screenshot displays the DVWA web application interface. At the top, the DVWA logo is visible. The main heading is "Vulnerability: DOM Based Cross Site Scripting (XSS)". Below this, there is a language selection box with "English" selected. A "More Information" section lists three links: <https://owasp.org/www-community/attacks/xss/>, https://owasp.org/www-community/attacks/DOM_Based_XSS, and <https://www.acunetix.com/blog/articles/dom-xss-explained/>. On the left, a sidebar menu lists various vulnerabilities, with "XSS (DOM)" highlighted. At the bottom, the user's session information is shown: "Username: admin", "Security Level: Security Level: high", and "Locale: en". There are also "View Source" and "View Help" buttons.

DVWA

Vulnerability: DOM Based Cross Site Scripting (XSS)

Please choose a language:
English Select

More Information

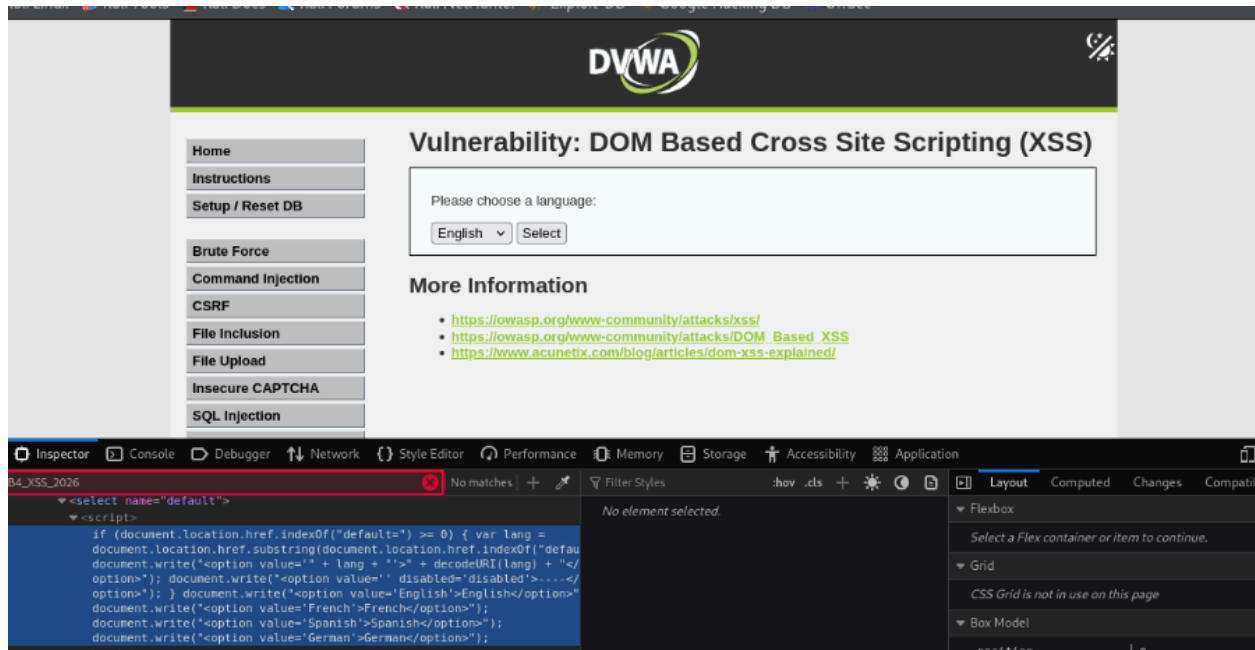
- <https://owasp.org/www-community/attacks/xss/>
- https://owasp.org/www-community/attacks/DOM_Based_XSS
- <https://www.acunetix.com/blog/articles/dom-xss-explained/>

Navigation Menu:

- Home
- Instructions
- Setup / Reset DB
- Brute Force
- Command Injection
- CSRF
- File Inclusion
- File Upload
- Insecure CAPTCHA
- SQL Injection
- SQL Injection (Blind)
- Weak Session IDs
- XSS (DOM)**
- XSS (Reflected)
- XSS (Stored)
- CSP Bypass
- JavaScript Attacks
- Authorisation Bypass
- Open HTTP Redirect
- Cryptography
- API
- DVWA Security
- PHP Info
- About
- Logout

Username: admin
Security Level: Security Level: high
Locale: en

[View Source](#) [View Help](#)



Phase 5: DVWA Stored XSS

Baseline run and then a safe retest

With Low security, the script marker was sent in the Stored XSS guestbook. It then showed up on later page views in the entry list without escaping. It stayed there on each load, which shows persistence and a real execution risk that goes past the first request.

https://127.0.0.1:4280/vulnerabilities/xss_s/

Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB OffSec

DVWA

Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

JavaScript Attacks

Authorisation Bypass

Vulnerability: Stored Cross Site Scripting (XSS)

Name *

Message *

Sign Guestbook

Clear Guestbook

Name: test
Message: This is a test comment.

Name: hello
Message:

More Information

- <https://owasp.org/www-community/attacks/xss>
- <https://owasp.org/www-community/xss-filter-evasion-cheatsheet>
- https://en.wikipedia.org/wiki/Cross-site_scripting
- <https://www.cgisecurity.com/xss-faq.html>
- <https://www.scriptalert1.com/>

https://127.0.0.1:4280/vulnerabilities/xss_s/

Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB OffSec

DVWA

Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

JavaScript Attacks

Authorisation Bypass

Vulnerability: Stored Cross Site Scripting (XSS)

Name *

Message *

Sign Guestbook

Clear Guestbook

Name: test
Message: This is a test comment.

Name: hello
Message:

Name: LAB4_XSS_2
Message: LAB4_XSS_2026

More Information

- <https://owasp.org/www-community/attacks/xss>
- <https://owasp.org/www-community/xss-filter-evasion-cheatsheet>
- https://en.wikipedia.org/wiki/Cross-site_scripting
- <https://www.cgisecurity.com/xss-faq.html>
- <https://www.scriptalert1.com/>

At High security, a plain control entry, "POSITIVE / I AM GOOD", was saved and shown as expected. When the script marker was submitted again at Impossible security, it was stored and displayed as escaped plain text. It did not run. Alongside it, the control entry "Impossible / IS THERE A BUG?" also appeared. Together these results point to output encoding as the key control, not a rule that blocks the input in advance.

DVWA

Home
Instructions
Setup / Reset DB
Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)
XSS (Stored)
CSP Bypass
JavaScript Attacks
Authorisation Bypass
Open HTTP Redirect
Cryptography
API
DVWA Security
PHP Info
About
Logout

Vulnerability: Stored Cross Site Scripting (XSS)

Name *
Message *
[Sign Guestbook](#) [Clear Guestbook](#)

Name: test
Message: This is a test comment.

Name: hello
Message:

Name: LAB4_XSS_2
Message: LAB4_XSS_2026

Name: POSTIVE
Message: I AM GOOD

More Information

- <https://owasp.org/www-community/attacks/xss>
- <https://owasp.org/www-community/xss-filter-evasion-cheatsheet>
- https://en.wikipedia.org/wiki/Cross-site_scripting
- <https://www.cgisecurity.com/xss-faq.html>
- <https://www.scriptalert1.com/>

Username: admin
Security Level: Security Level: high
Locale: en

[View Source](#) [View Help](#)



- Home
- Instructions
- Setup / Reset DB
- Brute Force
- Command Injection
- CSRF
- File Inclusion
- File Upload
- Insecure CAPTCHA
- SQL Injection
- SQL Injection (Blind)
- Weak Session IDs
- XSS (DOM)
- XSS (Reflected)
- XSS (Stored)**
- CSP Bypass
- JavaScript Attacks
- Authorisation Bypass
- Open HTTP Redirect
- Cryptography
- API
- DVWA Security
- PHP Info
- About
- Logout

Vulnerability: Stored Cross Site Scripting (XSS)

Name *

Message *

[Sign Guestbook](#) [Clear Guestbook](#)

Name: test
Message: This is a test comment.

Name: hello
Message: <script>alert('LAB4_XSS_2026')</script>

Name: LAB4_XSS_2
Message: LAB4_XSS_2026

Name: POSITIVE
Message: I AM GOOD

Name: impossible
Message: IS THERE A BUG?

More Information

- <https://owasp.org/www-community/attacks/xss>
- <https://owasp.org/www-community/xss-filter-evasion-cheatsheet>
- https://en.wikipedia.org/wiki/Cross-site_scripting
- <https://www.cgisecurity.com/xss-faq.html>
- <https://www.scriptalert1.com/>

Username: admin
Security Level: Security Level: impossible
Locale: en

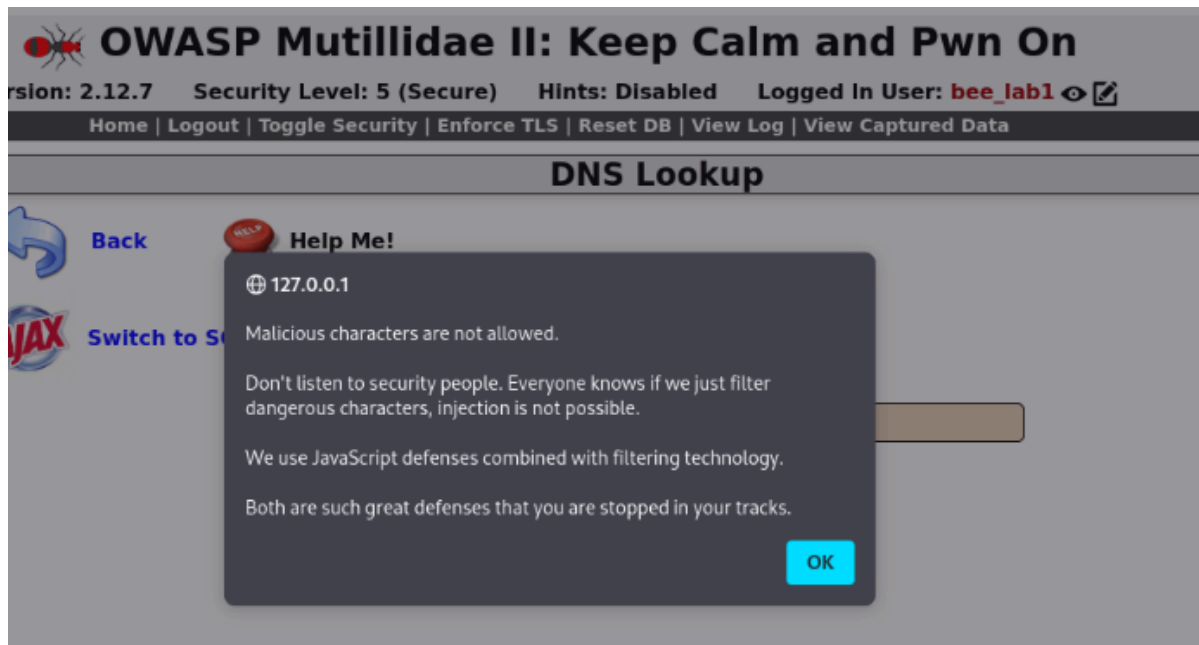
[View Source](#) [View Help](#)

Phase 6: Mutillidae Reflected XSS, then a safe retest

In Mutillidae at Security Level 0, labeled Hosed, the test input went in through the DNS Lookup Hostname/IP box. The browser then ran it, and an alert popped up with LAB4_XSS_2026.



When the setup was moved to Security Level 5, the same input did not get through. Instead, the app showed a message before anything was saved: “Malicious characters are not allowed.” This points to a block in the client side input handling, not a fix done by output encoding.

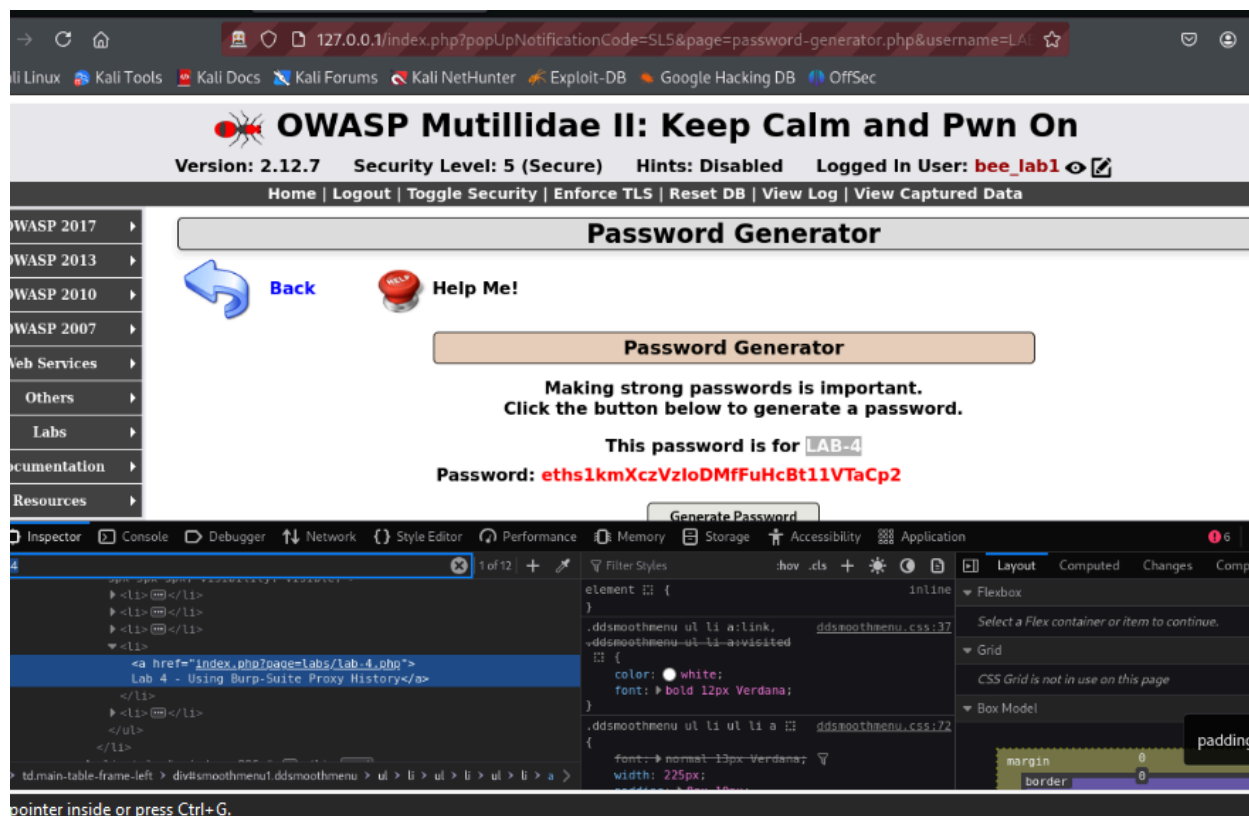
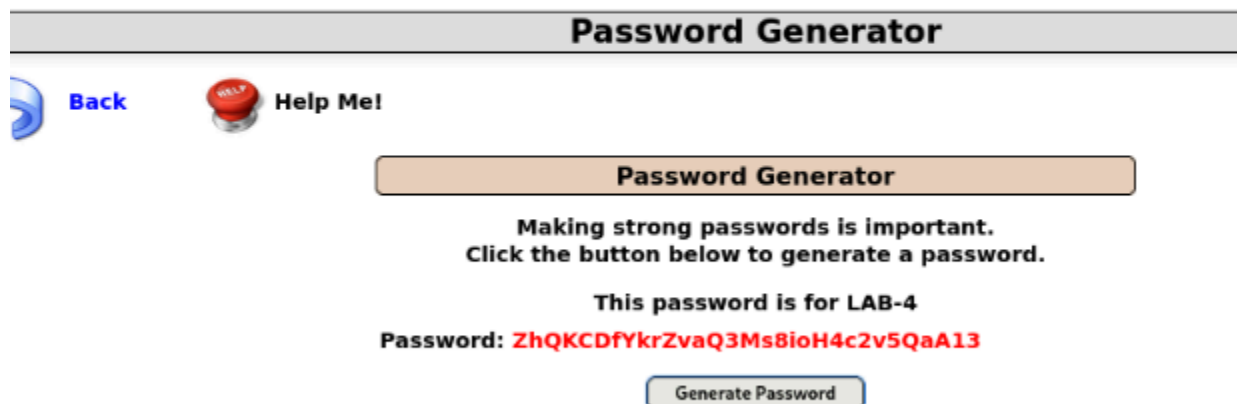
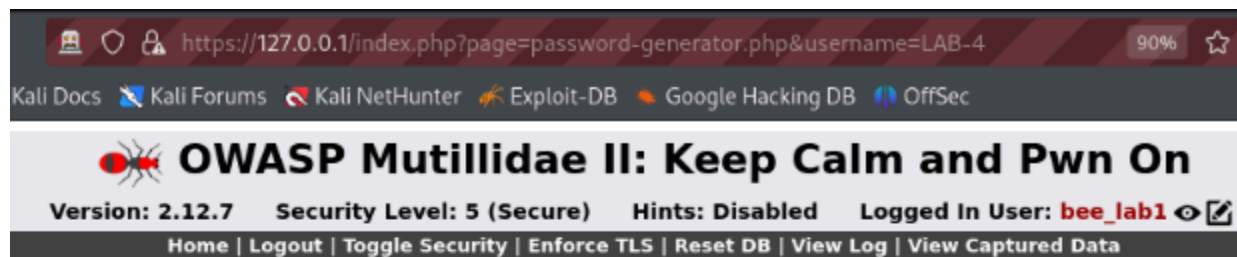


Phase 7: Mutillidae DOM XSS, Password Generator page

The Password Generator page takes the username GET value and places it into the on page title. It does this using client side script, so it acts as a DOM based source and sink, similar to the DVWA DOM XSS check. The test was run two ways. First, a fixed value was used: LAB4_DOM_2026. Second, the logged in user's real name was used: bee_lab1. In both runs, the browser Inspector showed the parameter value in the rendered DOM.



became a short string, “This password is for LAB-4,” instead of the full value. This suggests the higher mode adds length limits or filtering for that field.



Phase 8: Mutillidae Stored XSS — baseline check, fix retest, and a second login check

At Security Level 0, I tried to submit the script marker to the “Add to Your Blog” option. It did not save. The app showed a MySQL syntax error tied to the INSERT INTO blogs_table query. The cause was the unescaped single quote inside the payload, which broke the SQL.

Kali Linux

Kali Tools

Kali Docs

Kali Forums

Kali NetHunter

Exploit-DB

Google Hacking DB

OffSec

 **OWASP Mutillidae II: Keep Calm and Pwn On**

Version: 2.12.7 Security Level: 0 (Hosed) Hints: Enabled Logged In User: **bee_lab1** 

[Home](#) | [Logout](#) | [Toggle Hints](#) | [Toggle Security](#) | [Enforce TLS](#) | [Reset DB](#) | [View Log](#) | [View Captured Data](#)

OWASP 2017

OWASP 2013

OWASP 2010

OWASP 2007

Web Services

Others

Labs

Documentation

Resources

 **Back**

 **Help Me!**

 **Hints and Videos**

View Blog Entries

 **Add To Your Blog**

Select Author and Click to View Blog

Please Choose Author View Blog Entries

1 Current Blog Entries

	Name	Date	Comment
1	bee_lab1	2026-09-02 22:51:05	LAB4-STORED-2026

Want to Help?

 **Video Tutorials**

 **Announcements**

 **OWASP Mutillidae II: Keep Calm and Pwn On**

Version: 2.12.7 Security Level: 0 (Hosed) Hints: Enabled Logged In User: **bee_lab1** 

[Home](#) | [Logout](#) | [Toggle Hints](#) | [Toggle Security](#) | [Enforce TLS](#) | [Reset DB](#) | [View Log](#) | [View Captured Data](#)

 **Back**

 **Help Me!**

 **Hints and Videos**

Add New Blog Entry

 **View Blogs**

Add blog for bee_lab1

Note: , <i> and <u> are now allowed in blog entries

Save Blog Entry

 **View Blogs**

2 Current Blog Entries

	Name	Date	Comment
1	bee_lab1	2026-09-02 23:01:07	TEST AT LOW
2	bee_lab1	2026-09-02 22:51:05	LAB4-STORED-2026

CSRF Protection Information

This suggests the form mixes user input into both SQL and HTML. I did not run deeper SQL injection testing here. That was outside the lab goals, so it was not pursued.

To test storage in a safer way, I used a plain text marker: LAB4-STORED-2026. With Security Level 5 set, the save worked. The page states that only , , and are allowed. After that, the entry showed up under View Blogs.

 **OWASP Mutillidae II: Keep Calm and Pwn On**

Version: 2.12.7 Security Level: 5 (Secure) Hints: Disabled Logged In User: **bee_lab1**  

Home | Logout | Toggle Security | Enforce TLS | Reset DB | View Log | View Captured Data

Welcome To The Blog

 **Back**  **Help Me!**

Add New Blog Entry

 **View Blogs**

Add blog for bee_lab1

Note: , <i> and <u> are now allowed in blog entries

Save Blog Entry

 **View Blogs**

1 Current Blog Entries			
	Name	Date	Comment
1	bee_lab1	2026-09-02 22:51:05	LAB4-STORED-2026

 **OWASP Mutillidae II: Keep Calm and Pwn On**

Version: 2.12.7 Security Level: 5 (Secure) Hints: Disabled Logged In User: **bee_lab1**  

Home | Logout | Toggle Security | Enforce TLS | Reset DB | View Log | View Captured Data

View Blogs

 **Back**  **Help Me!**

View Blog Entries

 **Add To Your Blog**

Select Author and Click to View Blog

Please Choose Author View Blog Entries

1 Current Blog Entries			
	Name	Date	Comment
1	bee_lab1	2026-09-02 22:51:05	LAB4-STORED-2026

I then confirmed it stayed saved across sessions. I opened View Blogs in a separate private browser window with no login. The same entry was still visible.

Next, I went back to Security Level o. Two items showed up then. The earlier marker was still there, plus a new low-security item labeled “TEST AT LOW”. This matched what I needed to show: stored data remains even after later security-level changes.

Phase 9: DVWA session flow (sign in and sign out)

Before login, DVWA set a PHPSESSID cookie with the value f5b9f4d99838e8486a721c06aa470aco. After a successful login POST, that cookie value was checked again. The Set-Cookie result matched the earlier value. That means DVWA keeps the same session id after authentication.

The screenshot shows a web browser's developer tools interface. The left pane displays the 'Sites' tab with a list of contexts and sites. The right pane shows the 'Request' tab for a GET request to `http://127.0.0.1:4280/login.php`. The request headers are visible, including the `Cookie` header which contains `security=low; PHPSESSID=f5b9f4d99838e8486a721c06aa470aco; showhints=1; username=bee_lab1; uid=42`. The bottom pane shows the 'History' tab with a list of requests.

ID	Source	Req. Timestamp	Method	URL	Code	Reason	RTT	Size Resp. Body	Highest Alert	Note	Tags
399	Pr...	03/09/2026, 00:05:12	POST	http://127.0.0.1:4280/security.php	302	Found	1...	0 bytes	Informati...	SetCookie	
400	Pr...	03/09/2026, 00:05:13	GET	http://127.0.0.1:4280/security.php	200	OK	1...	5,257 bytes		AntiCSRF, Form...	
401	Pr...	03/09/2026, 00:05:31	GET	http://127.0.0.1:4280/logout.php	302	Found	3...	0 bytes			
402	Pr...	03/09/2026, 00:05:31	GET	http://127.0.0.1:4280/login.php	200	OK	5...	1,388 bytes		AntiCSRF, Form...	

Header: Text | Body: Text

```

HTTP/1.1 200 OK
Date: Wed, 02 Sep 2026 23:05:31 GMT
Server: Apache/2.4.68 (Debian)
X-Powered-By: PHP/8.5.9
Expires: Tue, 23 Jun 2009 12:00:00 GMT
Cache-Control: no-cache, must-revalidate
Pragma: no-cache
Vary: Accept-Encoding
Content-Length: 1388
Keep-Alive: timeout=5, max=99
Connection: Keep-Alive
Content-Type: text/html; charset=utf-8

```

```

<!DOCTYPE html>
<html lang="en-GB">
<head>

```

ID	Source	Req. Timestamp	Method	URL	Code	Reason	RTT	Size Resp. Body	Highest Alert	Note	Tags
399	Pr...	03/09/2026, 00:05:12	POST	http://127.0.0.1:4280/security.php	302 Found	1...	0 bytes		Information...	SetCookie	
400	Pr...	03/09/2026, 00:05:13	GET	http://127.0.0.1:4280/security.php	200 OK	1...	5,257 bytes			AntiCSRF, Form...	
401	Pr...	03/09/2026, 00:05:31	GET	http://127.0.0.1:4280/logout.php	302 Found	3...	0 bytes				
402	Pr...	03/09/2026, 00:05:31	GET	http://127.0.0.1:4280/login.php	200 OK	5...	1,388 bytes			AntiCSRF, Form...	

Then logout was tested. The server sent a 302 redirect to login.php. After that, index.php was requested using the same cookie that was present before logout. That request still ended at login.php. This shows the server stopped the session, not just the browser clearing it.

Header: Text | Body: Text

```

POST http://127.0.0.1:4280/login.php HTTP/1.1
host: 127.0.0.1:4280
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Content-Type: application/x-www-form-urlencoded
Content-Length: 88
Origin: https://127.0.0.1:4280
Connection: keep-alive
Referer: https://127.0.0.1:4280/login.php
Cookie: security=low; PHPSESSID=f5b9f4d99838e8486a721c06aa470ac0; showhints=1; username=bee_lab1; uid=42
Upgrade-Insecure-Requests: 1
Sec-Fetch-Dest: document
Sec-Fetch-Mode: navigate
Sec-Fetch-Site: same-origin
Sec-Fetch-User: 21

```

```

username=admin&password=password&Login=Login&user_token=14dd7e836629e82779bceec5a1b4171a

```

Quick Start → Request ← Response Requester +

Header: Text Body: Text

```
HTTP/1.1 302 Found
Date: Wed, 02 Sep 2026 23:08:39 GMT
Server: Apache/2.4.68 (Debian)
X-Powered-By: PHP/8.5.9
Set-Cookie: PHPSESSID=f5b9f4d99838e8486a721c06aa470ac0; expires=Thu, 03 Sep 2026 23:08:39 GMT; Max-Age=86400; path=/; domain=127.0.0.1
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: no-store, no-cache, must-revalidate
Pragma: no-cache
Location: index.php
Content-Length: 0
Keep-Alive: timeout=5, max=100
Connection: Keep-Alive
Content-Type: text/html; charset=UTF-8
```

Quick Start → Request ← Response Requester +

Header: Text Body: Text

```
GET http://127.0.0.1:4280/logout.php HTTP/1.1
host: 127.0.0.1:4280
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Connection: keep-alive
Referer: https://127.0.0.1:4280/vulnerabilities/authbypass/
Cookie: security=low; PHPSESSID=f5b9f4d99838e8486a721c06aa470ac0; showhints=1; username=bee_lab1; uid=42
Upgrade-Insecure-Requests: 1
Sec-Fetch-Dest: document
Sec-Fetch-Mode: navigate
Sec-Fetch-Site: same-origin
Sec-Fetch-User: ?1
Priority: u=0, i
```

Quick Start → Request ← Response Requester +

Header: Text Body: Text

```
HTTP/1.1 302 Found
Date: Wed, 02 Sep 2026 23:13:11 GMT
Server: Apache/2.4.68 (Debian)
X-Powered-By: PHP/8.5.9
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: no-store, no-cache, must-revalidate
Pragma: no-cache
Location: login.php
Content-Length: 0
Keep-Alive: timeout=5, max=100
Connection: Keep-Alive
Content-Type: text/html; charset=UTF-8
```

Phase 10: DVWA login strength

Five wrong password attempts were made in a row for the same DVWA test account. Each try returned the same login page and the same error message. No account lockout appeared. No step up delay was seen. No CAPTCHA was shown, and no other rate control behaviour was observed.

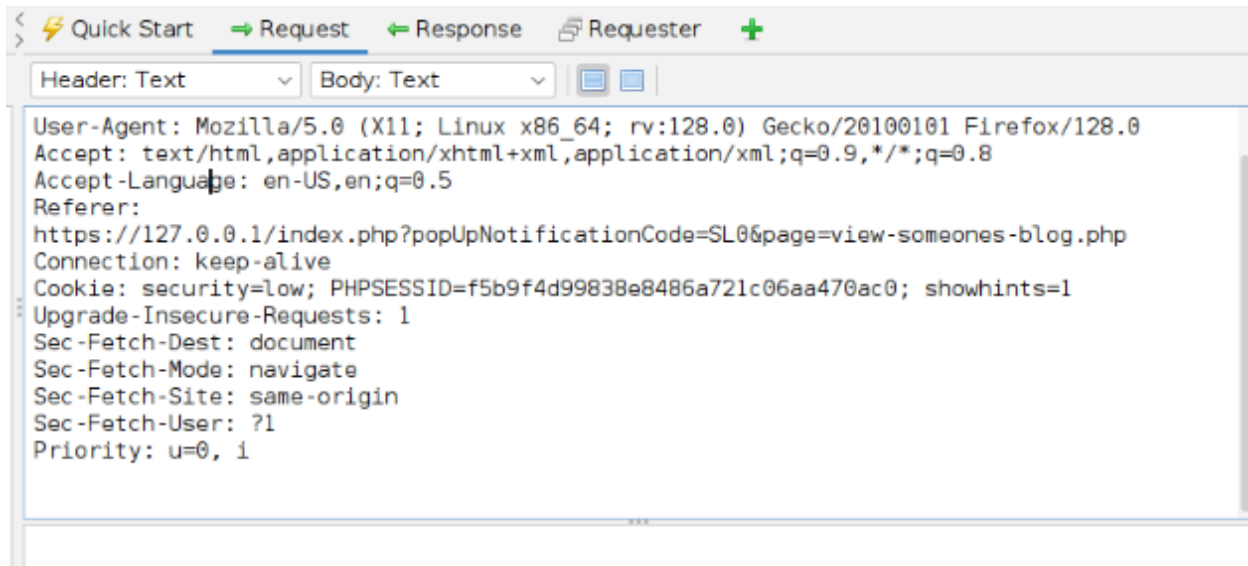
Phase 11: Mutillidae session flow and cookie checks

In the login POST, the PHPSESSID value stayed the same after the server replied. That means the system did not rotate the session during login. This matches what we saw in DVWA.

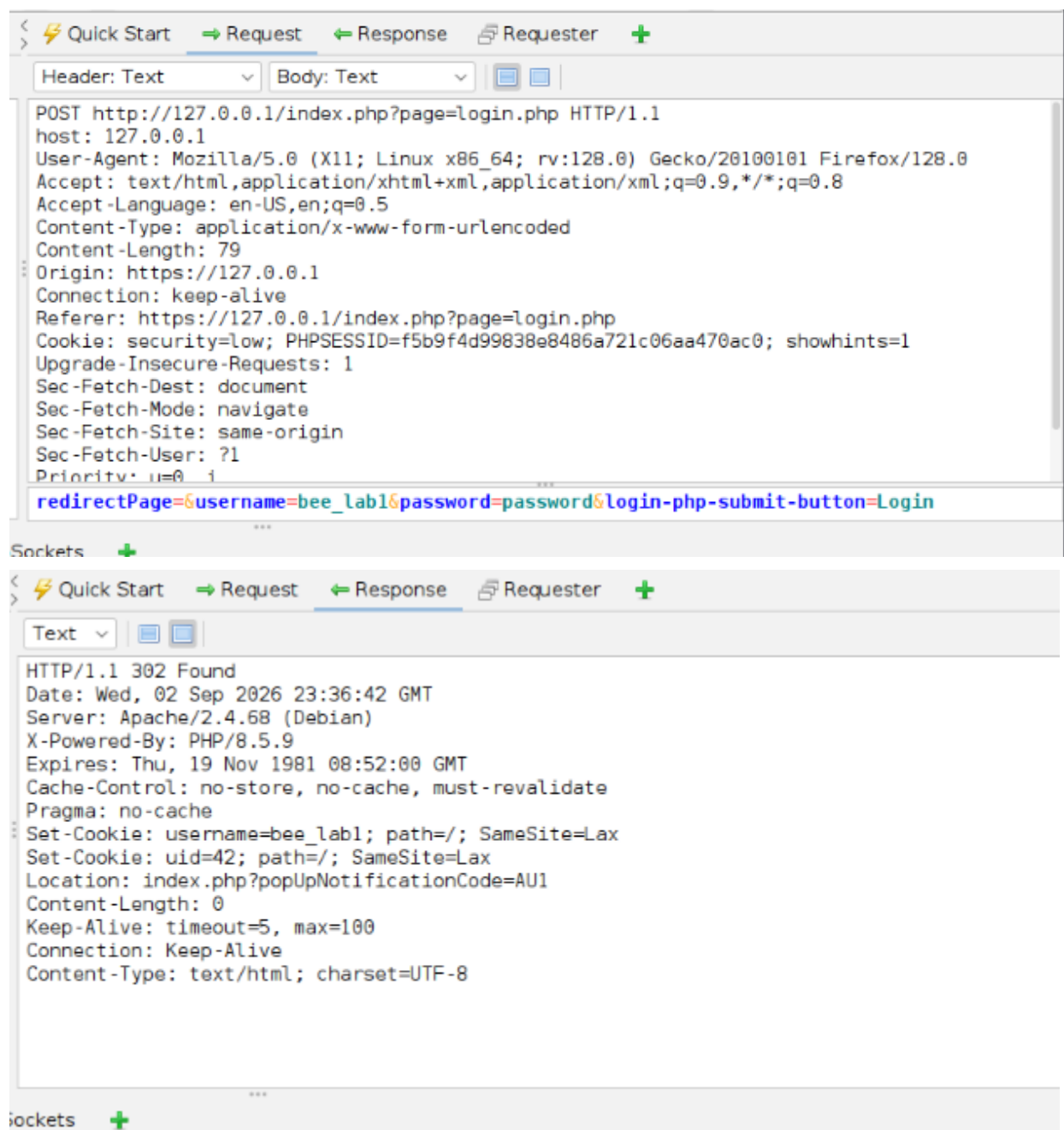
The screenshot displays the 'Request' tab of a web browser's developer tools. The 'Request' tab is selected, showing the details of an outgoing HTTP request. The 'Header' and 'Body' sections are both set to 'Text'. The request is a GET request to the URL `https://127.0.0.1/index.php?popUpNotificationCode=SL0&page=view-someones-blog.php`. The request headers include `Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8`, `Accept-Language: en-US,en;q=0.5`, `Connection: keep-alive`, `Referer: https://127.0.0.1/index.php?popUpNotificationCode=SL0&page=view-someones-blog.php`, `Cookie: security=low; PHPSESSID=f5b9f4d99838e8486a721c06aa470ac0; showhints=1; username=bee_lab1; uid=42`, `Upgrade-Insecure-Requests: 1`, `Sec-Fetch-Dest: document`, `Sec-Fetch-Mode: navigate`, `Sec-Fetch-Site: same-origin`, `Sec-Fetch-User: ?1`, and `Priority: u=0, 1`. The response tab is also visible, showing the status `HTTP/1.1 302 Found` and various headers including `Date: Wed, 02 Sep 2026 23:25:18 GMT`, `Server: Apache/2.4.68 (Debian)`, `X-Powered-By: PHP/8.5.9`, `Expires: Thu, 19 Nov 1981 08:52:00 GMT`, `Cache-Control: no-store, no-cache, must-revalidate`, `Pragma: no-cache`, and several `Set-Cookie` headers for `username=deleted`, `uid=deleted`, and `PHPSESSID=f5b9f4d99838e8486a721c06aa470ac0`. The `Location` header is `index.php?page=home.php&popUpNotificationCode=L0U1` and the `Content-Length` is `241`.

```
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Connection: keep-alive
Referer: https://127.0.0.1/index.php?popUpNotificationCode=SL0&page=view-someones-blog.php
Cookie: security=low; PHPSESSID=f5b9f4d99838e8486a721c06aa470ac0; showhints=1; username=bee_lab1; uid=42
Upgrade-Insecure-Requests: 1
Sec-Fetch-Dest: document
Sec-Fetch-Mode: navigate
Sec-Fetch-Site: same-origin
Sec-Fetch-User: ?1
Priority: u=0, 1
```

```
HTTP/1.1 302 Found
Date: Wed, 02 Sep 2026 23:25:18 GMT
Server: Apache/2.4.68 (Debian)
X-Powered-By: PHP/8.5.9
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: no-store, no-cache, must-revalidate
Pragma: no-cache
Set-Cookie: username=deleted; expires=Wed, 02 Sep 2026 22:25:18 GMT; Max-Age=0; path=/; SameSite=Lax
Set-Cookie: uid=deleted; expires=Wed, 02 Sep 2026 22:25:18 GMT; Max-Age=0; path=/; SameSite=Lax
Set-Cookie: uid=deleted; expires=Wed, 02 Sep 2026 22:25:18 GMT; Max-Age=0; path=/; HttpOnly; SameSite=Strict
Set-Cookie: username=deleted; expires=Wed, 02 Sep 2026 22:25:18 GMT; Max-Age=0; path=/; HttpOnly; SameSite=Strict
Location: index.php?page=home.php&popUpNotificationCode=L0U1
Content-Length: 241
Keep-Alive: timeout=5, max=100
```

For logout, the server did more than end the state. It set Max-Age=0 for the username and uid cookies, so those cookies were expired on purpose. The cookies showed up in two forms. One had no HttpOnly flag and used SameSite=Lax. The other used HttpOnly and SameSite=Strict. After logout, the next request did not show any sign of an active logged in state. I also checked the Set-Cookie headers from the login response. The username and uid cookies used SameSite=Lax. They did not include Secure, and they did not include HttpOnly. During this test, neither app ever showed a Secure attribute on any cookie.



Phase 12: Mutillidae login resilience

I sent five bad login tries to the same Mutillidae test account. Every time, the reply said "Password incorrect". I saw no rate limit behavior. I also saw no slow down after repeated tries, no lockout, and no CAPTCHA.

Phase 13: Cleanup and Restoration

Lab 4 test files were cleared. Any safe XSS check strings were removed too, or reset if needed. I signed out of the DVWA and Mutillidae accounts. Then I closed the test browser window and made sure it was emptied. I put the application security controls back to the default lab setup.

For logins, I did only the five planned failed tries for each target. I did not try anything else. I kept the needed screenshots, the request and response logs, and the proof files in the protected evidence folder. After that, the lab was left ready and clean, with all changes rolled back.

Cookie-Attribute Review

Attributes below are drawn from the Set-Cookie headers captured across the curl, Burp Suite and browser evidence for both applications.

Application	Cookie	Secure	HttpOnly	SameSite	Domain / Path	Expiry
DVWA	PHPSESSID	Not observed	Yes	Strict	127.0.0.1 / /	Max-Age=86400 (~24h)
DVWA	security	Not observed	Yes	Not observed	— / /	No Max-Age observed
Mutillidae	PHPSESSID	Not observed	Not observed	Not observed	— / /	Not observed
Mutillidae	showhints	Not observed	Not observed	Lax	— / /	Not observed
Mutillidae	username / uid	Not observed	Mixed (Lax form: No; Strict form: Yes)	Lax and Strict forms both seen	— / /	Max-Age=0 on logout

During this review, we never saw any cookie in either app that had the Secure flag. Only the Mutillidae cookies showed up. They were a username cookie and a uid cookie, and they appeared in two different setup styles.

Execution Context and Observed-versus-Potential Impact

Observed impact reflects only what was confirmed using harmless marker data. Potential impact is inferred from the same weakness and was not tested.

Application / Module	Affected parameter	Observed impact	Potential impact (not tested)
DVWA — Reflected & DOM XSS	name (GET), default (GET)	Script marker executed via browser alert at Low; encoded to literal text at Impossible.	An attacker-crafted link could execute arbitrary script in a victim's authenticated session (e.g. cookie/token theft, forced actions).
DVWA — Stored XSS	Guestbook Name / Message	Marker persisted and rendered unescaped for all viewers at Low; encoded at Impossible.	A stored script would execute for every user who views the guestbook, without a crafted link — broader and more persistent than reflected XSS.
Mutillidae — Reflected, DOM & Stored XSS	target_host, username (GET), blog comment	Marker executed via alert at Level 0; blocked at Level 5; the blog field also returned a SQL error on the script payload.	Same script-execution risk as DVWA; the SQL error suggests unsanitised input also reaches the database layer, a separate risk not exercised here.
DVWA & Mutillidae — Session Management	PHPSESSID	Session identifier unchanged across login in both apps; logout correctly invalidated the session server-side in both.	A session identifier obtained before login (e.g. via a shared network or fixation) would remain valid after the victim authenticates.
DVWA & Mutillidae — Cookie Attributes	PHPSESSID, security, username, uid	No cookie observed with the Secure attribute; HttpOnly and SameSite applied inconsistently.	Session or auth cookies sent over an insecure channel, or accessible to client-side script, are more exposed to interception or theft via XSS.

DVWA & Mutillidae — Authentication Resilience	username / password (POST)	Five consecutive failed attempts produced no lockout, delay or CAPTCHA on either target.	An attacker could attempt unlimited password guesses against a known username without being slowed or blocked.
---	----------------------------	--	--

Security-Level/Mode Comparison Summary

Test	Low / insecure setting	High / secure setting	Change observed
DVWA reflected XSS	Script executed via alert	Output HTML-encoded; rendered as text	Encoding applied at render, not input rejection
DVWA DOM XSS	Marker written into DOM directly	No marker written; parameter decoded/validated first	Client-side script logic changed to validate before use
DVWA stored XSS	Marker stored and executed on every view	Marker stored but encoded/escaped on display	Encoding applied to stored output as well as reflected output
Mutillidae reflected XSS	Script executed via alert	Rejected client-side before submission: "Malicious characters are not allowed"	Combined JavaScript + filtering blocks the payload pre-submission
Mutillidae stored XSS	Script marker broke the SQL insert (error, not stored)	Plain marker accepted; limited-tag allowance confirmed	Server-side handling changed markedly between levels
Session rotation (both apps)	Unchanged	Unchanged	No improvement observed at any tested level
Authentication resilience (both apps)	No lockout/delay after 5 attempts	No lockout/delay after 5 attempts	No improvement observed — resilience was not tied to the XSS security-level setting

Remediation Recommendations

- Encode output based on where it lands. If it goes in HTML text, use the right HTML entity encoding. If it goes in an HTML attribute, encode for that case. If it ends up inside inline JavaScript, encode for the JS string rules too. Do this at every output point. Do not rely on a special “secure demo” switch.
- Keep the same checks and the same encoding rules in every environment. Do not treat a weaker mode as acceptable just because a person can turn on a higher level later. A design that is unsafe by default is still unsafe.
- Change the session identifier whenever the user’s privilege level changes. Do this at login as well. Then stop using the old id on the server by invalidating it there.
- Set cookie flags for session and auth cookies. Use Secure and HttpOnly. Set SameSite to Strict, or to Lax only when you truly need cross site GET flows.
- Make logout actually kill the session on the server. Also enforce short session lifetimes on the server side. Relying only on cookie expiry is not enough.
- After a few wrong login attempts, slow things down or block further tries. You can use rate limits, progressive delay, or an account lockout. Use a monitored and safe process to unlock. If you need extra friction, add CAPTCHA or MFA.

Analysis Questions

1. What distinguishes reflected, stored and DOM XSS?

Reflected XSS shows up right away in what the server sends back. Stored XSS gets kept on the server, then hits anyone who opens the affected page later. DOM XSS stays in the browser and comes from bad JavaScript that mixes user input into the page in an unsafe way.

2. Why is a harmless visual marker enough to prove browser execution?

A visible marker proves that attacker-controlled input was interpreted as code or unescaped markup. A harmless payload is enough to demonstrate the vulnerability without using a harmful payload.

3. Why should a session rotate at login and become invalid after logout?

Rotating the session value when a user logs in helps stop session fixation. Ending or clearing the session at logout keeps a leaked session id from being used again.

4. How can account lockout itself create availability risk?

An attacker may try the login form over and over with bad passwords. If this locks out the real user, the lockout rule can be turned into a denial of service risk.

Challenges Faced

- With the deadline being so short, it was not realistic to do deep checks for every weakness and testing method. So the work focused on the items needed for the lab setup and on fixing problems that came up while running the tests.
- DVWA and Mutillidae did not behave the same way. They had different setups, different URL paths, separate login steps, and different ways of holding a session. Because of that, each app had to be checked in real time. Guessing that one workflow would match the other would have been risky.
- The DOM based XSS work took more than simple page checks. It also meant looking at what the browser built in the live DOM and how the JavaScript acted. The text you can view with basic page source often did not show what was needed as proof. In those cases, the browser tools had to be used, including the Inspector.
- Since the review needed screenshots of what was sent and what came back, plus cookies, DOM parts, login attempts, and session changes, the test runs had to be done with care. Each outcome also had to be saved in a form that could be used as evidence later.

Conclusion

The test was set up to check a claim from a development team. The claim was that DVWA and Mutillidae have enough browser-facing session and login controls. For both apps, the lowest security level was used first. In that mode, reflected, DOM-based, and stored XSS were each confirmed. The payload used was a harmless marker. No app rotated the session ID at login. Neither app set the Secure flag on its cookies. Then the security level was increased. This shift changed what happened with XSS. DVWA relied on encoding, and Mutillidae used filtering. Even so, two things stayed the same in both apps. Session rotation did not happen at login. The apps also showed no improvement in how they handled repeated login attempts. Mutillidae also had SQL errors shown in its blog write-up. That detail points to an input-handling flaw that goes beyond XSS. This looks similar to the earlier review of upload and command execution. In both cases, the security level switch changes behavior but does not fix the deeper design. The steps listed earlier focus on the core issues: output encoding in the right context, safer DOM methods, session ID rotation and session invalidation, using the Secure cookie attribute, and adding limits for authentication attempts.